Lab 4.1: Goroutines and Channels

lab4.1.go

```go
package main

import (
    "fmt"
    "time"
)

type Result struct {
    Task   string
    Values []int
}

func squares(n int, ch chan<- Result) {
    fmt.Println("Goroutine 'squares' started")
    time.Sleep(300 * time.Millisecond)
    var vals []int
    for i := 1; i <= n; i++ {
        vals = append(vals, i*i)
    }
    fmt.Println("Goroutine 'squares' completed")
    ch <- Result{"Squares", vals}
}

func cubes(n int, ch chan<- Result) {
    fmt.Println("Goroutine 'cubes' started")
    time.Sleep(100 * time.Millisecond)
    var vals []int
    for i := 1; i <= n; i++ {
        vals = append(vals, i*i*i)
    }
    fmt.Println("Goroutine 'cubes' completed")
    ch <- Result{"Cubes", vals}
}

func fibonacci(n int, ch chan<- Result) {
    fmt.Println("Goroutine 'fibonacci' started")
    time.Sleep(200 * time.Millisecond)
    vals := []int{0, 1}
    for i := 2; i < n; i++ {
        vals = append(vals, vals[i-1]+vals[i-2])
    }
    fmt.Println("Goroutine 'fibonacci' completed")
    ch <- Result{"Fibonacci", vals[:n]}
}

func main() {
    fmt.Println("Main: launching goroutines")
    ch := make(chan Result)

    go squares(8, ch)
    go cubes(8, ch)
    go fibonacci(8, ch)

    for i := 0; i < 3; i++ {
        r := <-ch
        fmt.Printf("Main received %s: %v\n", r.Task, r.Values)
    }
    fmt.Println("Main: all tasks finished")
}
```

Lab 4.1: Output

